Chapter 3
Algorithms

Section 3.3 : Complexity of Algorithms

Complexity of Algorithms

- Few Questions
  - When does an algorithm provide a satisfactory solution to a problem?
    - First, it must always produce the correct answer.
    - Second, it should be efficient.
  - How can the efficiency of an algorithm be analyzed?
    - One measure of efficiency is the time used by a computer to solve a problem using the algorithm.
    - A second measure is the amount of computer memory required to implement the algorithm when input values are of a specified size.

Complexity of Algorithms (Contd.)

- Questions such as these involve the computational **complexity of the algorithm**.
  - An analysis of the time required to solve a problem of a particular size involves the **time complexity** of the algorithm.
  - An analysis of the computer memory required involves the **space complexity** of the algorithm.
- Why study complexity of algorithm?
  - It is obviously important to know whether an algorithm will produce an answer in a microsecond, a minute, or a billion years. Hence, we need to take **time complexity** into account.
  - Likewise, the required memory must be available to solve a problem, meaning, **space complexity** must be taken into account.
- We will focus on **time complexity** in this course.

Time Complexity

- Can be expressed in terms of the number of operations used by the algorithm when the input has a particular size.
- Why consider only "$number\ of\ operations$" and not "$actual\ clock\ cycle$"?
  - Because of the difference in time needed for different computers to perform basic operations.
  - Moreover, it is quite complicated to break all operations down to the basic bit operations that a computer uses.
  - Furthermore, the fastest computers in existence can perform basic bit operations (for instance, $adding, multiplying, comparing$, or $exchanging\ two\ bits$) in $10^{-11} second$ $(10\ picoseconds)$, but personal computers may require $10^{-8}\ second$ $(10\ nanoseconds)$, which is 1000 times as long, to do the same operations.

Time Complexity (Contd.)

- MEANWHILE CPU ….

Time Complexity of Algorithms

- Example 1:

  Describe the time complexity of the Algorithm for finding the maximum element in a finite set of integers.

ALGORITHM 1 Finding the Maximum Element in a Finite Sequence.

procedure $max(a_1, a_2, \ldots, a_n$: integers)
$max := a_1$
for $i := 2$ **to** n
    if $max < a_i$ **then** $max := a_i$
return max {max is the largest element}

Time Complexity of Algorithms (Contd.)

- Solution:
  - $Basic\ Operations$: Comparisons.
  - $Measure\ of\ Time\ Complexity$: The number of comparisons .
  - To find the maximum element of a set with n elements, listed in an arbitrary order,
    - The temporary maximum is first set equal to the initial term in the list.
    - Then, the procedure is continued, using 2 additional comparisons for each term of the list
      - Firstly, $i \leq n$, to determine if the end of the list has been reached.
      - Secondly, $\max < a_i$, to determine whether to update the temporary maximum.
  - Basic comparisons,
    - 2 comparisons for each of the 2^{nd} through the n^{th} elements
    - 1 more comparison to exit the loop when $i = n + 1$.
  - So, exactly $2(n-1)+1 = 2n-1$ comparisons are used.
  - Hence, this algorithm has time complexity $\Theta(n)$, measured in terms of the number of comparisons used.

Note that for this algorithm the number of comparisons is independent of particular input of n numbers.

Time Complexity of Algorithms (Contd.)

- Worst-Case Complexity:
  - Worst-case analysis tells us the largest number of operations an algorithm requires to guarantee that it will produce a solution.

- Average-Case Complexity:
  - Average-case analysis tells us the number of operations used to solve a problem over all possible inputs of a given size.
  - Usually much more complicated than worst-case analysis.

Time Complexity of Algorithms (Contd.)

- Example 2:

  Describe the time complexity of the linear search algorithm

ALGORITHM 2 The Linear Search Algorithm.

procedure *linear search*(x: integer, $a_1, a_2, \ldots, a_n$: distinct integers)
$i := 1$
while ($i \leq n$ and $x \neq a_i$)
    $i := i + 1$
if $i \leq n$ **then** *location* $:= i$
else *location* $:= 0$
return *location*{*location* is the subscript of the term that equals x, or is 0 if x is not found}

Time Complexity of Algorithms (Contd.)

- Solution:
  - $Basic\ Operations$: Comparisons.
  - $Measure\ of\ Time\ Complexity$: The number of comparisons .
  - Comparisons
    - At each step of the loop in the algorithm, 2 comparisons are performed,
      - Firstly, $i \leq n$, to determine if the end of the list has been reached.
      - Secondly, $x < a_i$, to compare the element x with a term of the list.
    - Finally, 1 more comparison, $i \leq n$ outside the loop to return location.
  - So, for an element in the list, i.e. $x = a_i$, then $2i + 1$ comparisons are used.
  - And, for an element not in the list, i.e. $\forall i\ x \neq a_i$, in the worst-case, $2n + 2$ comparisons are used where,
    - $2n$ comparisons determine that $x \neq a_i$.
    - 1 comparison exits the loop.
    - 1 comparison outside the loop returns location.
  - Hence, this algorithm has time complexity $\Theta(n)$ in the **worst-case**, as $2n + 2$ is $\Theta(n)$.

Time Complexity of Algorithms (Contd.)

- Example 3:

  Describe the average-case performance of the linear search algorithm in terms of the average number of comparisons used, assuming that the integer x is in the list and it is equally likely that x is in any position.

ALGORITHM 2 The Linear Search Algorithm.

procedure *linear search*(x: integer, $a_1, a_2, \ldots, a_n$: distinct integers)
$i := 1$
while ($i \leq n$ and $x \neq a_i$)
    $i := i + 1$
if $i \leq n$ **then** *location* $:= i$
else *location* $:= 0$
return *location*{*location* is the subscript of the term that equals x, or is 0 if x is not found}

Time Complexity of Algorithms (Contd.)

- Solution:

  Comparisons-

  - At each step of the loop in the algorithm, 2 comparisons are performed,
    - Firstly, $i \leq n$, to determine if the end of the list has been reached.
    - Secondly, $x \neq a_i$, to compare the element x with a term of the list.
  - Finally, 1 more comparison, $i \leq n$ outside the loop to return location.

  So, for an element in the list, i.e. $x = a_i$, then $2i + 1$ comparisons are used.
  Therefore, total comparisons, for $i = 1,2,3 \ldots n$ using the relation $2i + 1$ is

$$\begin{aligned} & 3 + 5 + 7 + \cdots + (2n + 1) \\ &= (2 \cdot 1 + 1) + (2 \cdot 2 + 1) + (2 \cdot 3 + 1) + \cdots + (2 \cdot n + 1) \\ &= 2(1 + 2 + 3 + \cdots + n) + n \\ &= \frac{2n(n+1)}{2} + n \\ &= n(n + 2) \end{aligned}$$

  Thus, the average number of comparisons,

$$\frac{n(n + 2)}{n} = (n + 2)$$

  Hence, this algorithm has time complexity $\Theta(n)$ in the **average-case**, as $n + 2$ is $\Theta(n)$.

Time Complexity of Algorithms (Contd.)

- NOTES:
  - In the analysis of Example 3 we assumed that x is in the list being searched. It is also possible to do an average-case analysis of this algorithm when x may not be in the list
  - Although we have counted the comparisons needed to determine whether we have reached the end of a loop, these comparisons are often not counted. From this point on we will ignore such comparisons.

Algorithmic Paradigms

- Commonly used terminologies

TABLE 1 Commonly Used Terminology for the Complexity of Algorithms.

Complexity	*Terminology*
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Algorithmic Paradigms (Contd.)

- Brute Force Algorithm:
  - An important, and basic, algorithmic paradigm.
  - A problem is solved in the most straightforward manner based on the statement of the problem and the definitions of terms.
  - Designed to solve problems without regard to the computing resources required.
  - Although often inefficient, are often quite useful.

- Tractable:
  - A problem that is solvable using an algorithm with polynomial worst-case complexity, because the expectation is that the algorithm will produce the solution to the problem for reasonably sized input in a relatively short time.

Algorithmic Paradigms (Contd.)

- P vs NP:
  - Many solvable problems are believed to have the property that no algorithm with polynomial worst-case time complexity solves them.
  - But problems for which a solution, if known, can be checked in polynomial time are said to belong to the $\boldsymbol{class\ NP}$ $(Non-deterministic\ Polynomial\ time)$.
  - Tractable problems are said to belong to $\boldsymbol{class\ P}$.
- $NP-complete$:
  - If any problem can be solved by a polynomial worst-case time algorithm, then all problems in the $\boldsymbol{class\ NP}$ can be solved by polynomial worst-case time algorithms.

THE END